C/C++プログラムのSBOM作成・脆弱性確認 手順書（OSS版）

Yasuhiro Yokobori

2026-10-31

- はじめに
  - 前提
  - 作業の全体像
  - 本書の構成
  - 使うツール
  - 用語
- 1. ビルドPCの作業（Linux）
  - 1.1 用意するもの
  - 1.2 ネット接続PCでのダウンロード
  - 1.3 ビルドPCへの持ち込み
  - 1.4 作業1 ビルドで使った部品を調べる
  - 1.5 作業2 ビルド環境のOSパッケージを記録する
  - 1.6 作業PCへ持ち出す
  - チェックリスト
- 2. ビルドPCの作業（Windows）
  - 2.1 用意するもの
  - 2.2 作業1 ビルドで使った部品を調べる
  - 2.3 作業2 部品の出どころ（自動で分類されます）
  - 2.4 作業PCへ持ち出す
  - チェックリスト
- 3. 作業PCの準備
  - 3.1 用意するもの
  - 3.2 ネット接続PCでのダウンロード
  - 3.3 作業PCへのインストール
  - チェックリスト

- 4. 作業PCでの共通調査
  - 4.1 ビルドPCから運んだファイルを置く
  - 4.2 作業3 同梱ソースを調べる
  - 4.3 作業4 成果物の部品を調べる
- 5. ソフトウェア製品のSBOMを作る
  - 5.1 納入ファイルを確定する
  - 5.2 部品台帳を作る
  - 5.3 照合、変換、形式チェック
- 6. システム製品のSBOMを作る
  - 6.1 納入構成を記録する
  - 6.2 部品台帳を作る
  - 6.3 照合、変換、形式チェック
- 7. 脆弱性チェック
  - 7.1 作業6 脆弱性を照合する
  - チェックリスト
- 8. 定期作業と結果の判断
  - 8.1 いつ何をするか
  - 8.2 脆弱性データベースの更新
  - 8.3 結果の判断のしかた
  - 8.4 誤検知・影響なしの記録
  - 8.5 リリースごとに保管するもの
- 付録A 部品台帳の書き方
- 付録B CPEの調べ方
- 付録C 配布する実行ファイル
  - cpp_deps がしていること
- 付録D 困ったとき

はじめに

この手順書に従うと、C/C++で書かれた自社プログラムについて「どの外部部品（OSS）を使っているか」の一覧（SBOM）を作り、その部品に既知の脆弱性がないかを確認できます。使うツールはすべて無償のOSSと、本書の管理者が配布する実行ファイルで、**ソースコードや解析結果を社外に送ることは一切ありません。**

前提

作業には次の3種類のPCを使います。

PC	何をするか	入れるもの
ビルドPC	対象プログラムをビルドする。ここでビルドの記録を取り、結果を持ち出す。環境をきれいに保つため、ツールは最小限にする	cpp_deps、Syft（Linuxのみ。どちらもインストール不要）、bear（Linux で Make の場合のみ）
作業PC	インターネットにつながっていない Windows PC。ここで共通調査、製品種別ごとのSBOM作成、脆弱性照合を行う。製品が Linux 向けでも同じ	ScanCode、cpp_sbom、CycloneDX CLI、Grype、cve-bin-tool、Python、Excel
ネット接続PC	ツールと脆弱性データベースの**ダウンロードだけ**に使う。ソースコードは置かない	ダウンロード用のツールだけ

- コンテナ（Docker等）は対象外です。
- 対象はLinux向けプログラムとWindows向けプログラムです。ソフトウェアのみを納入する「ソフトウェア製品」と、OSを入れたサーバ・ワークステーションごと納入する「システム製品」で、製品SBOMの範囲を分けます。
- Windows向けは、Visual Studio のコンパイラ（MSVC）でビルドするものが対象です。MinGW や clang でビルドするものは対象外です。

作業の全体像

1. 対象のOSに応じて第1章または第2章でビルドを記録し、結果を作業PCへ運ぶ
2. 第3章で作業PCを準備し、第4章でソースと成果物を共通調査する
3. ソフトウェアだけを納入する場合は第5章、OS入りの機器を納入する場合は第6章で台帳とSBOMを作る
4. 第7章で脆弱性を照合し、第8章で定期的に更新・判断する

システム製品のOS情報は、**完成した納入構成**から取得します。LinuxのビルドPCで作る `build-env.cdx.json` はビルド環境の記録であり、納入OSの記録ではありません。

本書の構成

- 第1章：ビルドPCの作業（Linux）
- 第2章：ビルドPCの作業（Windows）
- 第3章：作業PCの準備
- 第4章：作業PCでの共通調査
- 第5章：ソフトウェア製品のSBOM

- 第6章：システム製品のSBOM
- 第7章：脆弱性チェック
- 第8章：定期作業と結果の判断
- 付録：台帳の書き方、CPEの調べ方、配布する実行ファイル、困ったとき

使うツール

ツール	何をするものか	使う場所
cpp_deps（本書の管理者から受け取る）	ビルドで使った部品を調べる（内容は付録C）	ビルドPC
bear	Make のビルドでコンパイルの記録を取る	ビルドPC（Linux で Make の場合のみ）
Syft	OSにインストールされた部品の一覧を作る	ビルドPC（Linux）・納入構成のLinux機器
ScanCode Toolkit	ソースフォルダ内のOSSとライセンスを見つける	作業PC
cpp_sbom（本書の管理者から受け取る）	部品台帳（Excel）を作り、記入後に完成を確かめ、SBOMに変換する	作業PC
CycloneDX CLI	SBOMファイルの結合と形式チェック	作業PC
Grype	SBOMと脆弱性データベースを突き合わせる	作業PC
cve-bin-tool	実行ファイルの中の部品を直接調べる	作業PC

用語

用語	意味
SBOM	ソフトウェアの部品表。使っている部品の名前・バージョン・ライセンスの一覧
CycloneDX	SBOMの代表的なファイル形式の一つ（JSON形式のテキストファイル）
脆弱性 / CVE	ソフトウェアのセキュリティ上の欠陥。CVEはその世界共通の識別番号（例：CVE-2024-12345）
CPE	製品を表す世界共通の名前。脆弱性データベースはこの名前で部品を探すため、**C/C++の部品ではCPEが正しく書けているかが照合の精度を左右します**
脆弱性データベース	既知の脆弱性の一覧。毎日更新されるため、定期的に取り込み直す必要がある
ターミナル	Linuxでコマンドを入力する画面。Windowsでは「PowerShell」が相当

本書のコマンドは、灰色の枠の中をそのままコピーして貼り付ければ動くように書いています。<> で囲んだ部分（例：<プロジェクトのフォルダ>）だけ、自分の環境に合わせて書き換えてください。

1. ビルドPCの作業（Linux）

対象プログラムをビルドするPC（Ubuntu / Debian 系、または Red Hat / Rocky / Alma 系。以下「ビルドPC」）で、作業1〜2を行います。コマンドは「ターミナル」に入力します。

ビルドPCの環境をきれいに保つため、**インストールするのは bear だけ**です（CMake でビルドしている場合は bear も不要です）。cpp_deps と Syft は、フォルダに置くだけで動きます。cpp_deps が何をするツールかは、付録Cに書いています。

1.1 用意するもの

用意するもの	入手先	備考
cpp_deps	本書の管理者から受け取る	`cpp_deps-linux-amd64`。インストール不要。ライセンス文（`THIRD_PARTY_LICENSES.txt`）も一緒に置く
Syft	GitHub: anchore/syft の Releases	`linux_amd64.tar.gz` を選ぶ（`.deb` や `.rpm` は使わない）。インストール不要
bear	OSの標準パッケージ	Make でビルドしている場合のみ。CMake なら不要

本書作成時点（2026年11月）の Syft の版は 1.38.0 です。より新しい版があればそちらを使ってください。

1.2 ネット接続PCでのダウンロード

(1) Syft を取得する。 ネット接続PCは Windows でも Linux でも構いません。ブラウザで次のURLを開いて保存します。

`https://github.com/anchore/syft/releases/download/v1.38.0/syft_1.38.0_linux_amd64.tar.gz`

(2) bear を取得する（Make でビルドしている場合のみ）

ビルドPCと**同じOS・同じバージョン**の Linux で実行します。

```
# Ubuntu / Debian 系
mkdir -p os-pkg && cd os-pkg && apt-get download $(apt-cache depends --recurse --no-recommends --no-suggests --no-
          conflicts --no-breaks --no-replaces --no-enhances bear | grep '^\w' | sort -u) ; cd ..
# Red Hat 系
dnf download --resolve --destdir os-pkg bear
```

この方法は、ネット接続PCとビルドPCの**OSの版と更新状態が同じ**ことが前提です。インストール時に「ダウングレード」という表示が出た場合は、作業を中止して開発担当者に相談してください。

(3) まとめて運ぶ

受け取った `cpp_deps-linux-amd64` と `THIRD_PARTY_LICENSES.txt`、取得した Syft の `.tar.gz`、`os-pkg` フォルダ（bear）を1つのフォルダにまとめ、社内規定に沿った方法（USBメモリ等）でビルドPCへ運びます。**このフォルダにはツールしか入っていないので、持ち込みで社内の情報が外に出ることはありません。**

1.3 ビルドPCへの持ち込み

運んだファイルを ~/sbom-tools に置いて、次を実行します。

```
mkdir -p ~/sbom-work
cd ~/sbom-tools
chmod +x cpp_deps-linux-amd64
mkdir -p syft && tar xzf syft_*_linux_amd64.tar.gz -C syft

# bear（Make の場合のみ）
sudo apt install ./os-pkg/*.deb        # Red Hat 系は sudo dnf install ./os-pkg/*.rpm
```

動作確認として、次がバージョンを表示すれば準備完了です。

```
~/sbom-tools/syft/syft version
```

1.4 作業1 ビルドで使った部品を調べる

プログラムを一度ビルドし直し、その途中で「コンパイルで読み込んだヘッダ」と「リンクで使ったライブラリ（静的ライブラリ .a を含む）」を記録します。そのあと cpp_deps で、各ファイルがどのOSパッケージのものかを調べ、結果を deps.json というファイルに書き出します。**ビルド手順そのものは開発チームの手順書に従ってください。** この作業だけ開発担当者に依頼しても構いません。

(1) 記録を取りながらビルドする

どちらの場合も、リンカに「使ったファイルを表示する」指定（-Wl,--trace）を加え、画面に出た内容を build_link.log に保存します。

Make でビルドしている場合：

```
cd <プロジェクトのフォルダ>
make clean
bear -- make CC="gcc -Wl,--trace" CXX="g++ -Wl,--trace" 2>&1 | tee build_link.log
```

CMake でビルドしている場合（bear は不要）：

```
cd <プロジェクトのフォルダ>
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON \
  -DCMAKE_EXE_LINKER_FLAGS="-Wl,--trace" -DCMAKE_SHARED_LINKER_FLAGS="-Wl,--trace"
cmake --build build --clean-first 2>&1 | tee build_link.log
```

compile_commands.json（CMake の場合は build フォルダの中）と build_link.log の2つができれば成功です。-Wl,--trace は記録を出すだけの指定で、できあがるプログラムの中身は変わりません。

(2) cpp_depsを実行する

```
~/sbom-tools/cpp_deps-linux-amd64 \
  --compdb compile_commands.json \
  --linklog build_link.log \
  --bin <実行ファイルや.soがあるフォルダ> \
  --src . \
  --out ~/sbom-work/deps.json
```

CMake でビルドしている場合は、--compdb を build/compile_commands.json にしてください。

最後に「注意:」で始まるメッセージが出た場合は、表示された内容に従ってビルドからやり直してください。**この調査はビルドPCでしかできないので、持ち出す前に解決します。** 特に「ビルドログにリンクの記録がありません」と出た場合は、Makefile がコンパイラ名を直接書いているなどの理由で指定が効いていません。開発担当者に、リンク時に `-Wl,--trace` が付くよう調整を依頼してください。「場所を特定できないライブラリが◯件あります」と出た場合は、ビルドをやり直さず、`--linkdir` に `build_link.log` に書かれた相対パスの基準フォルダを指定して、`cpp_deps` を再実行してください。リンクしたファイルをログから探し直します。再実行しても残ったものは、そのまま持ち出して構いません（台帳の行として残ります）。

(3) 結果を確かめる

`~/sbom-work/deps.json` ができ、「注意:」が出ていなければ成功です。このファイルは第5章または第6章でツールが読むもので、人が開く必要はありません。部品の区分や確認は、すべて第5章または第6章の台帳で行います。

この方法でも見つけられないもの： 他社や別部署からビルド済みの実行ファイル・DLLを受け取り、リンクせずにそのまま同梱している場合です。これは第4章のcve-bin-toolで検出します。

1.5 作業2 ビルド環境のOSパッケージを記録する

ビルドPCにインストールされているOSパッケージをすべて記録します。これは製品のSBOMとは別の**「ビルド環境の記録」**として保管し、後で「どの環境でビルドしたか」を説明するときに使います。

```
cd ~
SYFT_CHECK_FOR_APP_UPDATE=false SYFT_FILE_METADATA_SELECTION=none \
~/sbom-tools/syft/syft scan dir:/ \
  --override-default-catalogers "dpkg-db-cataloger,rpm-db-cataloger" \
  --exclude './proc/**' --exclude './sys/**' --exclude './dev/**' \
  --exclude './home/**' --exclude './tmp/**' \
  -o cyclonedx-json=$HOME/sbom-work/build-env.cdx.json
```

- `SYFT_...` で始まる2つの指定は、更新の確認で外部に通信しないようにするなどの設定です。設定ファイルは作らないので、ビルドPCには何も残りません。
- OS全体を対象にするため、プロジェクトのフォルダではなく `/`（OSのルート）を指定します。`/home` を除外しているのは、個人のファイルを記録に含めないためです。

実行中に `WARN no explicit name and version provided for directory source` などの警告が表示されることがありますが、結果には影響しません。数秒から数分で終わります。この記録には各OSパッケージのライセンス（OSの copyright ファイルに書かれたもの）も入っており、第5章または第6章で台帳を作るときに使います。

1.6 作業PCへ持ち出す

これ以降の作業は、別のWindows PC（以下「作業PC」）で行います。次の3つと、必要なら `delivery.tar.gz` を社内規定に沿った方法で作業PCへ運びます。

持ち出すもの	作り方	使う作業
`deps.json`	作業1でできたもの	第5章または第6章
`build-env.cdx.json`	作業2でできたもの	第5章または第6章
`source.tar.gz`	下のコマンドで作る。**ビルドした状態の**プロジェクトのフォルダ全体（ソースと成果物）をまとめたもの	作業3、第5章または第6章

持ち出すもの	作り方	使う作業
delivery.tar.gz（必要な場合）	プロジェクト外の、顧客へ渡すファイルを集めたフォルダ	第4章〜第6章

```
cd <プロジェクトのフォルダの1つ上のフォルダ>
tar czhf ~/sbom-work/source.tar.gz --exclude=.git <プロジェクトのフォルダ名>
```

- h は、シンボリックリンク（別のファイルを指す印）を実体のファイルに置き換える指定です。Windows ではシンボリックリンクをうまく展開できないことがあるためです。リンク先のないシンボリックリンクがあると File removed before we read it と表示されますが、そのリンクが入らないだけで、ほかのファイルはまとめられています。
- 実行ファイルや .so のフォルダがプロジェクトのフォルダの**外**にある場合は、そのフォルダも同じコマンドで別のファイル（例：artifacts.tar.gz）にまとめて運びます。
- ソースはビルドしたものと同じ版にしてください。ビルド後に更新したソースを運ぶと、SBOMが実際の製品と合わなくなります。
- 開発担当者が実際の納入ファイルを delivery フォルダに集め、プロジェクト外にある場合はその親フォルダで tar czf ~/sbom-work/delivery.tar.gz delivery を実行して運びます。追加のツールは不要です。

作業PCに運べたら、ビルドPCの ~/sbom-tools と ~/sbom-work は削除して構いません。これで cpp_deps と Syft は完全に消えます。bear も不要なら削除できます。**ただし、リリースごとに作業1からやり直すので、残しておくと次回の準備が省けます。**

チェックリスト

- [ ] 作業1：-Wl,--trace 付きでビルドし、cpp_deps 実行時に「注意:」が出ないことを確認した
- [ ] 作業1：deps.json を作成した
- [ ] 作業2：build-env.cdx.json を作成した
- [ ] deps.json、build-env.cdx.json、source.tar.gz を作業PCへ運んだ

次は第3章の作業PCの準備、第4章の共通調査に進み、納入形態に応じて第5章または第6章を実行します。

2. ビルドPCの作業（Windows）

対象プログラムをビルドするPC（Windows 10 / 11、Visual Studio がインストール済み。以下「ビルドPC」）で、作業1〜2を行います。コマンドは **Developer PowerShell for VS** に入力します。

対象は、Visual Studio のコンパイラ（MSVC）でビルドするプログラムです。 ソリューションのほか、CMake や nmake を使っていても構いません。MinGW や clang でビルドするものは、この手順では調べられません。

ビルドPCの環境をきれいに保つため、**持ち込むのは cpp_deps だけ**です。インストールは不要で、フォルダに置くだけで動きます。何をするツールかは、付録Cに書いています。

2.1 用意するもの

用意するもの	入手先	備考
cpp_deps	本書の管理者から受け取る	`cpp_deps-windows-amd64.exe`。ライセンス文（`THIRD_PARTY_LICENSES.txt`）も一緒に置く

受け取ったファイルを、社内規定に沿った方法でビルドPCの `C:\sbom-tools` に置きます。

Windows には、Linux のような「OSパッケージの一覧」の仕組みがないので、Syft は使いません。部品の出どころは、作業1のツールが**「Microsoft製」「プロジェクト内」「それ以外」に自動で分類します**。

2.2 作業1 ビルドで使った部品を調べる

プログラムを一度ビルドし直し、その途中で「コンパイルで読み込んだヘッダ」と「リンクで使ったライブラリ（静的ライブラリ `.lib` を含む）」をログに記録します。そのあと `cpp_deps` で、成果物のEXE/DLLと合わせて調べ、結果を `deps.json` というファイルに書き出します。**ビルド手順そのものは開発チームの手順書に従ってください。** この作業だけ開発担当者に依頼しても構いません。

(1) 「Developer PowerShell for VS」を開く

スタートメニューで「Developer PowerShell」と入力し、使っている Visual Studio の版のものを開きます。この画面では `msbuild` が使えます。結果を置く作業用フォルダも、ここで作っておきます。

```
New-Item -ItemType Directory -Force C:\sbom-work | Out-Null
```

(2) 記録を取りながらビルドする

ビルドのしかたによってコマンドが違います。どれにあたるか分からないときは、開発担当者に確認してください。

Visual Studio のソリューション（`.sln`）をビルドする場合：

```
cd <プロジェクトのフォルダ>
$env:VSLANG = "1033"
$env:_CL_ = "/showIncludes"
$env:_LINK_ = "/VERBOSE"
msbuild <ソリューション名>.sln /t:Rebuild /m:1 /p:Configuration=Release /p:Platform=x64 `
  /fl "/flp:logfile=C:\sbom-work\build.log;verbosity=normal"
Remove-Item Env:VSLANG, Env:_CL_, Env:_LINK_
```

CMake でビルドする場合（ビルド用のフォルダがすでに作られていること）：

```
cd <プロジェクトのフォルダ>
$env:VSLANG = "1033"
$env:_CL_ = "/showIncludes"
$env:_LINK_ = "/VERBOSE"
cmake --build <ビルド用のフォルダ> --config Release --clean-first -- /m:1 `
  /fl "/flp:logfile=C:\sbom-work\build.log;verbosity=normal"
Remove-Item Env:VSLANG, Env:_CL_, Env:_LINK_
```

CMake は、ビルド用のフォルダに .sln か .slnx のファイルがある場合（Visual Studio 用の設定）だけ、この方法で記録できます。ない場合は記録が取れないことがあるので、Visual Studio 用の設定でビルド用のフォルダを作り直すよう、開発担当者に依頼してください。

nmake（Makefile）でビルドする場合：

```
cd <Makefileのあるフォルダ>
$env:VSLANG = "1033"
$env:_CL_ = "/showIncludes"
$env:_LINK_ = "/VERBOSE"
nmake /a 2>&1 | Out-File -Encoding utf8 C:\sbom-work\build.log
Remove-Item Env:VSLANG, Env:_CL_, Env:_LINK_
```

- VSLANG は、コンパイラとリンカのメッセージを英語にする設定です。英語の言語パックが入っていない Visual Studio では日本語のままですが、ツールはどちらの言語でも読み取れます。
- _CL_ は「読み込んだヘッダを表示する」、_LINK_ は「使ったライブラリを表示する」設定です。最後の行で元に戻します。
- /m:1 は、1つずつ順番にビルドする指定です。並列にビルドするとログが混ざり、正しく読み取れないことがあるためです。
- Configuration、Platform、--config は、実際に出荷する設定に合わせてください。
- nmake の /a は、すべてをビルドし直す指定です。
- いずれも記録を出すための設定で、できあがるプログラムの中身は変わりません。ログは数十MBと大きくなることがあります。

(3) cpp_depsを実行する

同じ Developer PowerShell の画面で実行します。

```
C:\sbom-tools\cpp_deps-windows-amd64.exe `
  --bin <EXEやDLLが出力されたフォルダ> `
  --src <プロジェクトのフォルダ> `
  --buildlog C:\sbom-work\build.log `
  --out C:\sbom-work\deps.json
```

最後に「注意:」で始まるメッセージが出た場合は、表示された内容に従って (2) からやり直してください。**この調査はビルドPCでしかできないので、持ち出す前に解決します。** 「MSVC 以外のコンパイラ」と表示された場合は、やり直しても解決しません。この手順の対象外なので、本書の管理者に相談してください。

(4) 結果を確かめる

C:\sbom-work\deps.json ができ、「注意:」が出ていなければ成功です。このファイルは第5章または第6章でツールが読むもので、人が開く必要はありません。部品の区分や確認は、すべて第5章または第6章の台帳で行います。

2.3 作業2 部品の出どころ（自動で分類されます）

作業1の cpp_deps が、部品を出どころごとに自動で分類しています。**ここで人がする作業はありません。** 分類の結果は第5章または第6章の台帳に入り、確認と記入は台帳で行います。

MSVC と Windows SDK の情報は deps.json に記録されますが、ビルド用ツールとして製品台帳には自動で入れません。これらを納入構成に導入して渡すシステム製品では、第6章で納入構成を調べて台帳に入れます。

この方法でも見つけられないもの： 他社や別部署からビルド済みの実行ファイル・DLLを受け取り、リンクも読み込みもせずに同梱している場合です。これは第4章のcve-bin-toolで検出します。

2.4 作業PCへ持ち出す

これ以降の作業は、別のWindows PC（以下「作業PC」）で行います。次の2つと、必要なら delivery.tar.gz を社内規定に沿った方法で作業PCへ運びます。

持ち出すもの	作り方	使う作業
deps.json	作業1でできたもの	第5章または第6章
source.tar.gz	下のコマンドで作る。**ビルドした状態の**プロジェクトのフォルダ全体（ソースと成果物）をまとめたもの	作業3、第5章または第6章
delivery.tar.gz（必要な場合）	プロジェクト外の、顧客へ渡すファイルを集めたフォルダ	第4章〜第6章

```
cd <プロジェクトのフォルダの1つ上のフォルダ>
tar czf C:\sbom-work\source.tar.gz --exclude=.git --exclude=.vs <プロジェクトのフォルダ名>
```

- EXEやDLLのフォルダがプロジェクトのフォルダの**外**にある場合は、そのフォルダも同じコマンドで別のファイル（例：artifacts.tar.gz）にまとめて運びます。
- ソースはビルドしたものと同じ版にしてください。ビルド後に更新したソースを運ぶと、SBOMが実際の製品と合わなくなります。
- 開発担当者が実際の納入ファイルを delivery フォルダに集め、プロジェクト外にある場合はその親フォルダで tar czf C:\sbom-work\delivery.tar.gz delivery を実行して運びます。追加のツールは不要です。

作業PCに運べたら、ビルドPCの C:\sbom-tools と C:\sbom-work は削除して構いません。**ただし、リリースごとに作業1からやり直すので、残しておくと次回の準備が省けます。**

チェックリスト

- [ ] 作業1：記録を取りながらビルドし、cpp_deps 実行時に「注意:」が出ないことを確認した
- [ ]

作業1：deps.json を作成した

- [ ] deps.json、source.tar.gz を作業PCへ運んだ

次は第3章の作業PCの準備、第4章の共通調査に進み、納入形態に応じて第5章または第6章を実行します。

3. 作業PCの準備

台帳とSBOMの作成、脆弱性の確認は、インターネットにつながっていないWindows PC（以下「作業PC」。Windows 10 / 11）で行います。**製品が Linux 向けでも Windows 向けでも、作業PCでの手順は同じです。** コマンドは **PowerShell** に入力します。ツールの準備は最初に一度だけ行います。脆弱性データベースの定期的な更新は、第8章を見てください。

3.1 用意するもの

用意するもの	入手先	備考
Python 3	python.org	ScanCode と cve-bin-tool に必要。インストール時に「Add python.exe to PATH」にチェック
ScanCode Toolkit	GitHub: aboutcode-org/scancode-toolkit の Releases	`windows` と、Python の版（例：`py3.12`）が書かれた `.zip` を選ぶ
CycloneDX CLI	GitHub: CycloneDX/cyclonedx-cli の Releases	`cyclonedx-win-x64.exe` を選ぶ
Grype	GitHub: anchore/grype の Releases	`windows_amd64.zip` を選ぶ。**0.121.2 以上を使う**（古い版は Windows で脆弱性データベースの取り込みに失敗することがある）
Grype用 脆弱性データベース	grype.anchore.io（手順3.2で取得）	毎日更新される。定期的に取り直す（第8章）
cve-bin-tool	PyPI（pip）	Python 3 が必要。作業4で、実行ファイルの中の部品を調べるのに使う
cpp_sbom	本書の管理者から受け取る	`cpp_sbom-windows-amd64.exe` と `windows-system-inventory.ps1`。第5章または第6章で使う
Excel	社内の標準	台帳の記入に使う

本書作成時点（2026年11月）の版は ScanCode 32.5.0、Grype 0.121.2、CycloneDX CLI 0.30.0 です。より新しい版があればそちらを使ってください。

3.2 ネット接続PCでのダウンロード

ネット接続PCの PowerShell で実行します。Python は**作業PCと同じ版**をインストールしておいてください（cve-bin-tool の部品がPythonの版に依存するため）。

(1) 置き場所を作り、版を指定する

```
New-Item -ItemType Directory -Force C:\sbom-tools | Out-Null
cd C:\sbom-tools
$GRYPE_V = "0.121.2"
$CDX_V = "0.30.0"
```

(2) Grype と CycloneDX CLI を取得する

```
Invoke-WebRequest -Uri "https://github.com/anchore/grype/releases/download/v$GRYPE_V/grype_${GRYPE_V}
          _windows_amd64.zip" -OutFile grype.zip
Invoke-WebRequest -Uri "https://github.com/CycloneDX/cyclonedx-cli/releases/download/v$CDX_V/cyclonedx-win-x64.exe"
          -OutFile cyclonedx.exe
```

(3) Grype の脆弱性データベースを取得する

```
$info = Invoke-RestMethod https://grype.anchore.io/databases/v6/latest.json
$url = "https://grype.anchore.io/databases/v6/" + $info.path
$url
Invoke-WebRequest -Uri $url -OutFile ($info.path -replace ':', '_')
```

3行目で `vulnerability-db_v6...tar.zst` で終わるURLが表示され、4行目でそのファイルが保存されます。ファイル名に Windows で使えない文字（:）が含まれるので、_ に置き換えて保存します。

(4) ScanCode Toolkit と Python のインストーラを取得する

ブラウザで ScanCode の Releases ページを開き、最新版の **`windows` と Python の版を含む `.zip`** を `C:\sbom-tools` に保存します（作業PCの Python が 3.12 なら `py3.12-windows.zip`）。作業PCに Python がない場合は、python.org から同じ版のインストーラ（`python-3.x.x-amd64.exe`）も保存しておきます。

(5) cve-bin-tool と、その動作に必要なデータを取得する

```
py -m pip download cve-bin-tool -d cve-bin-tool-pkg
py -m pip install --user cve-bin-tool
py -m cve_bin_tool.cli --update now -n json-nvd -d "OSV,GAD"
py -m cve_bin_tool.cli --export cve-db.export
```

3行目はデータの取得で、10分ほどかかります。`-d "OSV,GAD"` は、エラーで止まるデータ取得元を外す指定です。途中の `Unable to fetch EPSS` というエラーと、最後の「対象フォルダが指定されていない」という趣旨のエラーは、想定どおりの動作です。

cve-bin-tool 3.4 は、現在の NVD（米国の脆弱性データベース）からデータを取得できません。**そのため本書では cve-bin-tool を部品の検出だけに使い、脆弱性の確認は作業6の Grype で行います。**

(6) 受け取った実行ファイルを置く

本書の管理者から受け取った `cpp_sbom-windows-amd64.exe`、`windows-system-inventory.ps1`、ライセンス文（`THIRD_PARTY_LICENSES.txt`）を `C:\sbom-tools` に置きます。Windowsシステム製品では、このスクリプトを完成した納入構成にコピーして実行します。

最後に `C:\sbom-tools` フォルダごと、社内規定に沿った方法で作業PCへ運びます。**このフォルダにはツールと公開データしか入っていないので、持ち込みで社内の情報が外に出ることはありません。**

3.3 作業PCへのインストール

運んだフォルダを作業PCの `C:\sbom-tools` に置き、PowerShell で実行します。ScanCode は長いパスの中に置くと初回設定に失敗するので、**必ず `C:\sbom-tools` の直下に展開してください。**

```
cd C:\sbom-tools
New-Item -ItemType Directory -Force C:\sbom-work | Out-Null

# Grype
Expand-Archive grype.zip -DestinationPath C:\sbom-tools\grype -Force

# ScanCode Toolkit（ファイル名は実際のものに置き換える）
Expand-Archive scancode-toolkit-*windows*.zip -DestinationPath C:\sbom-tools -Force

# cve-bin-tool
py -m pip install --user --no-index --find-links cve-bin-tool-pkg cve-bin-tool
```

ツールの場所を登録する。 毎回フォルダを指定しなくて済むよう、PowerShell を開くたびに次を実行します（ScanCode のフォルダ名は実際のものに置き換える）。

```
$env:Path = "C:\sbom-tools;C:\sbom-tools\grype;C:\sbom-tools\scancode-toolkit-<版>;" + $env:Path
```

ScanCode の初回設定。 初めての実行時に自動で設定が行われます。数分かかることがあります。ヘルプが表示されれば完了です。

```
scancode --help
```

ネットにつながない設定にする。 Grype が起動時に更新を確認しに行かないよう、設定ファイルを作ります。

```
@"
check-for-app-update: false
db:
  auto-update: false
  validate-age: false
"@ | Set-Content -Encoding ascii "$env:USERPROFILE\.grype.yaml"
```

脆弱性データベースを取り込む。

```
grype db import (Get-Item C:\sbom-tools\vulnerability-db_v6*.tar.zst).FullName
grype db status
New-Item -ItemType Directory -Force "$env:USERPROFILE\.cache\cve-bin-tool" | Out-Null
py -m cve_bin_tool.cli --import C:\sbom-tools\cve-db.export
```

1行目は数分かかります。`grype db status` で `Status: valid` と、取得した日付（Built）が表示されれば成功です。最後の行では、最後に「対象フォルダが指定されていない」という趣旨のエラーが出ますが、これは想定どおりの動作です。

動作確認。 次の3つがそれぞれバージョンを表示すれば準備完了です。

```
grype version
cyclonedx --version
py -m cve_bin_tool.cli --version
```

チェックリスト

- [ ] ツールを作業PCにインストールし、3つのツールのバージョン表示を確認した
- [ ] 脆弱性データベース（Grype、cve-bin-tool）を取り込んだ

準備が終わったら 4. SBOM作成 に進みます。

4. 作業PCでの共通調査

作業PCで作業3〜4を行います。次に、納入形態に応じて第5章または第6章に進みます。PowerShell を開いたら、はじめに 3.3 の「ツールの場所を登録する」コマンドを実行してください。付録（台帳の列の意味、CPE の調べ方、困ったとき）は、本書の末尾にあります。

4.1 ビルドPCから運んだファイルを置く

運んだファイルを C:\sbom-work に置き、source.tar.gz を展開します。製品ごとにフォルダを分ける場合は、以下の C:\sbom-work を読み替えてください。

製品	C:\sbom-work に置くもの
Linux 向け	deps.json、build-env.cdx.json、source.tar.gz
Windows 向け	deps.json、source.tar.gz

```
cd C:\sbom-work
tar xzf source.tar.gz
```

C:\sbom-work\<プロジェクトのフォルダ名> ができます。artifacts.tar.gz を運んだ場合は、同じように展開します。

実際に納入するファイルを C:\sbom-work\delivery に置きます。delivery.tar.gz を運んだ場合は C:\sbom-work で tar xzf delivery.tar.gz を実行します。プロジェクト内の delivery だけを運んだ場合は、そのフォルダを C:\sbom-work\delivery にコピーします。**ビルドフォルダ全体ではなく、顧客へ渡すファイルだけ**を入れてください。第5章・第6章の台帳と、下記の成果物調査で同じフォルダを使います。

4.2 作業3 同梱ソースを調べる

ソースフォルダの中にコピーされているOSS（third_party、external、lib などのフォルダに置かれていることが多い）を ScanCode で調べます。プロジェクトのフォルダ全体を調べるので、OSSがどのフォルダにあるかを前もって確かめる必要はありません。

```
scancode -clpi -n 4 `
  --json-pp C:\sbom-work\scancode.json `
  --html C:\sbom-work\scancode.html `
  C:\sbom-work\<プロジェクトのフォルダ名>
```

- -clpi は「著作権表示・ライセンス・パッケージ情報・ファイル情報を調べる」という意味です。
- -n 4 は4並列で処理するという意味です。PCの性能に合わせて増減できます。
- プロジェクトの大きさによっては時間がかかります。

scancode.json は第5章または第6章でツールが読みます。フォルダごとのライセンスと著作権者、ヘッダから読み取ったバージョン候補が台帳に入るので、ここで内容を書き換える必要はありません。

検出されたライセンスが、そのOSSの公式のライセンスと違って見えることがあります。たとえば検証に使った nlohmann-json（公式は MIT）では「MIT AND Apache-2.0」と検出されました。これは json.hpp の中に Google Abseil（Apache-2.0）から取り込んだコードが含まれているためで、検出結果のほうが実態を表しています。**公式のライセンス名だけを根拠に検出結果を消さず、該当ファイルと著作権表示を確認してください。**

詳しく見たい場合は、scancode.html をブラウザで開くと、ファイルごとのライセンスと著作権表示を確認できます。

4.3 作業4 成果物の部品を調べる

ビルドPCから運んだ実行ファイルを cve-bin-tool で直接調べ、中に埋め込まれている部品を確かめます。Linux 向けの実行ファイルや .so も、作業PC（Windows）のまま調べられます。結果は、第5章または第6章で cpp_sbom check が台帳と照らし合わせ、台帳の漏れを見つけます。**ここで調べるのは部品だけです。脆弱性は第7章で確認します。**

```
py -m cve_bin_tool.cli --offline `
  --sbom-output C:\sbom-work\cvebin-parts.json --sbom-type cyclonedx --sbom-format json `
  C:\sbom-work\delivery
```

データが取り込まれていないと、画面に「offline operation」を案内するメッセージが出て、結果ファイルは作られません。cvebin-parts.json ができていることを必ず確認してください。

調査結果がそろったら、ソフトウェア製品は第5章、システム製品は第6章に進みます。

5. ソフトウェア製品のSBOMを作る

この章は、OSを含むサーバやワークステーションを納入せず、ソフトウェアだけを納入する場合に使います。第1章または第2章でビルドを記録し、第3章の準備と第4章の共通調査を終えてから実行します。**ビルドPCのOS全体は製品SBOMに入れません。**

5.1 納入ファイルを確定する

第4章で用意した `C:\sbom-work\delivery` に、顧客へ渡すファイルがそろっていることを確認します。Linuxの `.so` やWindowsのDLLを同梱する場合も、このフォルダに入れます。

`deps.json` はビルドで使用したファイルの記録、`delivery` は納入するファイルの記録です。この二つの違いから、ビルドPCにだけある動的ライブラリを製品部品から外します。照合にはファイル名を使うため、同名で版が異なるファイルがある場合は台帳で実物を確認してください。**静的ライブラリと、ヘッダだけの部品は製品にコードが入る可能性があるため、台帳で確認します。**

5.2 部品台帳を作る

Linux向けソフトウェアの結果を処理する場合：

```
cd C:\sbom-work
cpp_sbom-windows-amd64.exe ledger --product software `
  --deps deps.json --scan scancode.json --osenv build-env.cdx.json `
  --src C:\sbom-work\<プロジェクトのフォルダ名> `
  --delivery C:\sbom-work\delivery --out 部品台帳.xlsx
```

Windows向けソフトウェアの場合：

```
cd C:\sbom-work
cpp_sbom-windows-amd64.exe ledger --product software `
  --deps deps.json --scan scancode.json `
  --src C:\sbom-work\<プロジェクトのフォルダ名> `
  --delivery C:\sbom-work\delivery --out 部品台帳.xlsx
```

`build-env.cdx.json` はLinuxのビルド環境のライセンス確認用で、納入OSの調査結果ではありません。

台帳には納入物に含まれる候補と、判断が必要なヘッダ・自社ビルドのライブラリが出ます。区分が空の行を確認し、自社コードだけなら「対象外」、OSSのコードが製品に入るなら「同梱」にします。ビルド環境にしかない動的ライブラリは台帳に出ません。実行時に顧客側OSへ必要なライブラリは、製品の動作要件として別に記録してください。

色の付いたセルの意味は付録Aを見てください。自動で入った名前、版、ライセンスも根拠が不明なら開発担当者に確認します。台帳を保存してExcelを閉じます。

5.3 照合、変換、形式チェック

```
cpp_sbom-windows-amd64.exe check --ledger C:\sbom-work\部品台帳.xlsx `
  --cvebin C:\sbom-work\cvebin-parts.json
cpp_sbom-windows-amd64.exe convert --ledger C:\sbom-work\部品台帳.xlsx `
  --name <製品名> --version <製品のバージョン> `
  --out C:\sbom-work\product.cdx.json
cyclonedx validate --input-file C:\sbom-work\product.cdx.json --fail-on-errors
```

check の「要確認」は第4章のバイナリ調査と台帳が一致しないことを示します。実物を確認して、行の追加・修正、または誤検出の理由を記入してください。「CPEもPURLもありません」は脆弱性を照合できない可能性がある部品です。**形式チェックに合格しても、脆弱性をすべて照合できたとは限りません。**

完成した product.cdx.json を第7章で照合します。リリースごとの保管物は第8章を見てください。

6. システム製品のSBOMを作る

この章は、ソフトウェアを入れたサーバまたはワークステーションを、OSごと納入する場合に使います。第1章または第2章でソフトウェアのビルドを記録し、第3章の準備と第4章の共通調査を終えてから実行します。**OSの情報はビルドPCではなく、完成した納入構成から取得します。**

6.1 納入構成を記録する

ソフトウェアの納入ファイルを `C:\sbom-work\delivery` に用意します（第5章5.1と同じ範囲）。さらに、完成した納入機器または納入イメージで、次のどちらかを行います。ツールは置いて実行するだけで、インストールしません。結果を社内規定に従って作業PCへ運びます。

Linuxの納入構成：第1章で入手したポータブル版Syftの `.tar.gz` を、納入構成を確認する機器に置きます。実際の納入構成で次を実行します。ビルドPCで作った `build-env.cdx.json` を流用しないでください。

```
mkdir -p ~/sbom-tools/syft ~/sbom-work
tar xzf syft_*_linux_amd64.tar.gz -C ~/sbom-tools/syft
SYFT_CHECK_FOR_APP_UPDATE=false SYFT_FILE_METADATA_SELECTION=none \
~/sbom-tools/syft/syft scan dir:/ \
  --override-default-catalogers "dpkg-db-cataloger,rpm-db-cataloger" \
  --exclude './proc/**' --exclude './sys/**' --exclude './dev/**' \
  --exclude './home/**' --exclude './tmp/**' \
  -o cyclonedx-json=$HOME/sbom-work/system-inventory.cdx.json
```

この結果はOSパッケージの台帳です。`/opt` やユーザー別の場所にパッケージ管理外で導入したソフトウェアがあれば、開発・構築担当者から納入構成の一覧を受け取り、後で台帳に行を追加してください。

Windowsの納入構成：配布物の `windows-system-inventory.ps1` を納入機器へ置き、PowerShellで実行します。Windows標準の機能で、OSの版、機器全体にインストールされたアプリの登録情報、更新プログラムを記録します。

```
powershell -NoProfile -File C:\sbom-tools\windows-system-inventory.ps1 `
  -OutputDirectory C:\sbom-work
```

`system-inventory.cdx.json` と `windows-updates.csv` を作業PCへ運びます。このスクリプトはユーザー別に導入されたアプリ、ポータブルアプリ、ドライバ・ファームウェアを網羅しません。納入構成の一覧と突き合わせ、抜けたソフトウェアは台帳に追加してください。`windows-updates.csv` は `Get-HotFix` で見える更新だけの記録です。更新履歴の全件とは限らないため、納入構成の更新管理記録と合わせて脆弱性を判断します。

6.2 部品台帳を作る

Linux向けシステム製品の場合：

```
cd C:\sbom-work
cpp_sbom-windows-amd64.exe ledger --product system `
  --deps deps.json --scan scancode.json --osenv build-env.cdx.json `
  --src C:\sbom-work\<プロジェクトのフォルダ名> `
  --delivery C:\sbom-work\delivery `
  --system-inventory C:\sbom-work\system-inventory.cdx.json `
  --out 部品台帳.xlsx
```

Windows向けシステム製品の場合：

```
cd C:\sbom-work
cpp_sbom-windows-amd64.exe ledger --product system `
  --deps deps.json --scan scancode.json `
  --src C:\sbom-work\<プロジェクトのフォルダ名> `
  --delivery C:\sbom-work\delivery `
  --system-inventory C:\sbom-work\system-inventory.cdx.json `
  --out 部品台帳.xlsx
```

`--system-inventory` は必ず**納入構成**で作ったファイルを指定します。`build-env.cdx.json` はLinuxのビルド環境のライセンス確認にだけ使います。

納入OSとそのパッケージ・アプリは台帳に入ります。ライセンスが調査結果にないOSパッケージは空欄のままで構いません。Windowsの登録済みアプリは会社名が台帳の参考列に入りますが、CPE/PURLは自動で付かないため、該当する行は照合識別子を調べて記入します。見つからなければ「なし」と記録し、脆弱性照合の対象外となることを判断記録に残します。製品本体が登録済みアプリとして出た行は、二重掲載しないよう区分を「対象外」にして理由を書きます。版が空の導入済みソフト、出どころが不明な部品、同梱ソースなどの色の付いたセルも確認してください。台帳を保存してExcelを閉じます。

6.3 照合、変換、形式チェック

```
cpp_sbom-windows-amd64.exe check --ledger C:\sbom-work\部品台帳.xlsx `
  --cvebin C:\sbom-work\cvebin-parts.json
cpp_sbom-windows-amd64.exe convert --ledger C:\sbom-work\部品台帳.xlsx `
  --name <システム製品名> --version <納入構成の版> `
  --out C:\sbom-work\product.cdx.json
cyclonedx validate --input-file C:\sbom-work\product.cdx.json --fail-on-errors
```

台帳の「作成条件」シートに `system` と記録され、`check` と `convert` は納入OSが1行あることを確認します。形式チェックは脆弱性照合の可否を保証しません。完成したSBOMを第7章で照合します。WindowsのOSと更新プログラムについては、Grypeの結果だけで判断せず、納入機器の更新記録とMicrosoftの修正情報も確認してください。

7. 脆弱性チェック

第5章または第6章で作ったSBOMを使って、作業PCで作業6を行います。**製品が Linux 向けでも Windows 向けでも、手順は同じです。** リリース後も、脆弱性データベースを更新して定期的にこの章の作業をやり直します。

7.1 作業6 脆弱性を照合する

```
cd C:\sbom-work
grype sbom:product.cdx.json -o table --file grype-result.txt
grype sbom:product.cdx.json -o json --file grype-result.json
```

`grype-result.txt` に、見つかった脆弱性が表形式で書かれます。主な列の意味は次のとおりです。結果の判断のしかたは 第8章を見てください。

列	意味
NAME / INSTALLED	部品名と、使っているバージョン
FIXED IN	データベースが示す修正版の候補。空欄でも修正版がないとは断定できない
VULNERABILITY	脆弱性の番号（CVE-…）
SEVERITY	深刻度（Critical ＞ High ＞ Medium ＞ Low）

何も表示されないときは、「脆弱性がない」のか「照合できていない」のかを区別する必要があります。第5章・第6章で表示された「CPEもPURLもありません」の部品を確認し、台帳のCPEやPURLが正しいか調べてください（付録B）。Windowsシステム製品のOS更新は、`windows-updates.csv` とMicrosoftの修正情報でも確認します。

チェックリスト

- [ ] 作業6：`grype-result.txt` を作成し、第8章の手順で判断した

結果の判断のしかた、誤検知の除外、リリースごとに保管するものは、第8章を見てください。

8. 定期作業と結果の判断

SBOMは一度作れば終わりではありません。脆弱性は毎日新しく見つかるため、**部品が変わらなくても、脆弱性データベースを新しくして照合し直す**必要があります。この章の作業は作業PCで行います（製品がLinux向けでもWindows向けでも同じです）。

8.1 いつ何をするか

タイミング	すること	該当する手順
毎週（推奨）、または出荷判定の前	脆弱性データベースを取り直し、保管済みのSBOMで作業6（照合）だけをやり直す	8.2
製品のリリースごと、部品を追加・更新したとき	作業1〜6をすべてやり直し、SBOMを作り直す	第1章または第2章、第4章と第5章または第6章、第7章
3か月に1回程度	ツール本体を新しい版に入れ替える	3.2〜3.3（ビルドPCのツールは 1.2〜1.3 または 2.1）

8.2 脆弱性データベースの更新

1. ネット接続PCで、3.2の手順 **(3)** だけを実行し、新しいデータベースファイル（`vulnerability-db_v6...tar.zst`）を取得する
2. 作業PCへ運ぶ
3. 作業PCで、3.3の「脆弱性データベースを取り込む」の、`grype` で始まる2行を実行する（古いデータは自動で置き換わる）
4. `grype db status` の Built の日付が新しくなったことを確認する
5. 保管済みの `product.cdx.json` に対して、第7章のコマンドを実行する

8.3 結果の判断のしかた

`grype-result.txt` に出た脆弱性は、1件ずつ次の順で判断し、結果を記録します。

1. **本当にその部品・バージョンか確認する。** CPEによる照合では、名前の似た別製品が当たる「誤検知」が起きることがあります。部品名とバージョンが台帳と一致しているかを確認します。
2. **深刻度を見る。** Critical と High を優先して扱います。
3. **修正版があるか見る。** FIXED IN 列の版は候補です。開発チームに、対象の配布元が示す修正情報と、更新後の版で影響が解消するか確認を依頼します。OSパッケージには修正を古い版へ取り込む場合があります。
4. **修正版がない、またはすぐ更新できない場合**は、開発チームに「その脆弱性が自社プログラムの使い方で実際に影響するか」の確認を依頼します。脆弱性の詳しい内容は、CVE番号で NVD や JVN iPedia を検索すると読めます（CVE番号だけを入力するので、社内の情報は外に出ません）。
5. **判断結果を記録する。** 次の表の形で、リリースごとに残します。

CVE番号	部品	深刻度	判断	対応	判断者	判断日
（例）CVE-2024-xxxxx	zlib 1.2.11	High	影響あり	1.3.1へ更新（次リリース）		

CVE番号	部品	深刻度	判断	対応	判断者	判断日
（例）CVE-2023-xxxxx	minilib 1.2.3	Medium	誤検知（別製品）	除外リストに登録		

8.4 誤検知・影響なしの記録

誤検知や影響なしと判断しても、Grype の共通設定から除外せず、8.3の記録表に対象製品・版、判断の根拠を残します。次のリリースで部品や使い方が変わったら再評価してください。

8.5 リリースごとに保管するもの

ファイル	内容
`部品台帳.xlsx`	部品の一覧（人が確認した正本）
`product.cdx.json`	製品のSBOM
`build-env.cdx.json`	ビルド環境の記録（Linuxのみ）
`system-inventory.cdx.json`	納入構成の記録（システム製品）
`windows-updates.csv`	納入構成の更新プログラム（Windowsシステム製品）
`grype-result.json`、`grype-result.txt`	脆弱性の照合結果
`cvebin-parts.json`	成果物での裏取り結果（見つかった部品の一覧）
判断記録（8.3の表）	脆弱性ごとの判断と対応

あわせて、照合に使った脆弱性データベースの日付（`grype db status` の Built）も記録しておくと、「いつ時点の情報で判断したか」を後から説明できます。

付録A 部品台帳の書き方

台帳は `cpp_sbom` が Excel ファイル（`部品台帳.xlsx`）として作るので、一から作る必要はありません。人が記入するのは、シート「部品台帳」の色の付いたセルです。色は、その行で必須の項目が空のときに付き、埋めると消えます。シート「調査結果」は確認用、「作成条件」は製品種別の記録です。

列	書き方
区分	一覧から選ぶ（次の表）
部品名	部品の名前。フォルダ名が入っている場合は、わかりやすい名前に直してよい
バージョン	部品のバージョン。値が入っていれば、正しいかを開発担当者に確認する
ライセンス	ScanCode の結果に出た表記（例：MIT、Apache-2.0、BSD-3-Clause）。誤りが確認できた場合は、根拠を備考に残して修正する
CPEベンダー / CPE製品名	付録Bの方法で調べる。調べてもなければ、CPEベンダーに「なし」と書く
PURL	OSパッケージの行には自動で入る。ほかの部品は正しい識別子が判明した場合に記入する
確認者	確認した人の名前。記入があれば、その行は確認済みとして扱う
備考	メモ。区分を「対象外」にした理由もここに書く
場所 / 著作権者 / バージョン候補 / やること	灰色の列。参考情報で、SBOMには入らない

区分	意味	必須の項目
OSパッケージ	納入構成のOSパッケージ（システム製品）	部品名、バージョン。ライセンスは判明したものを記録
同梱	プロジェクトの中にあるOSS（OSSを自社でビルドしたものも含む）	部品名、バージョン、ライセンス、CPE（ベンダーと製品名の両方。なければベンダーに「なし」）またはPURL、確認者
別途導入	OSパッケージ以外の方法で導入した部品	同梱と同じ
納入OS	納入するサーバ・ワークステーションのOS（システム製品）	部品名、バージョン
導入済みソフト	納入構成にインストールしたソフトウェア（システム製品）	部品名、バージョン、CPEまたはPURL。なければCPEベンダーに「なし」と記録
対象外	自社で書いたコードだけのものや、作業4の誤検出など、SBOMに載せないもの	確認者、備考（理由）

区分が空の行や、必須の項目が空の行が1つでもあると、`cpp_sbom convert` はSBOMを作りません。区分・部品名・バージョン・PURLが同じ行は、SBOMでは1つの部品にまとまるので、手で1行にまとめる必要はありません。OSパッケージの名前と版が同じでも、アーキテクチャの違うPURLは別の部品として残します。

付録B CPEの調べ方

脆弱性データベースは、部品を CPE という決まった名前で探します。台帳の「CPEベンダー」「CPE製品名」がこの名前と一致しないと、脆弱性があっても見つかりません。作業PCに取り込み済みの Grype のデータベースで、ネットにつながずに調べられます。

```
grype db search --pkg <部品名>
```

例えば `grype db search --pkg libpng` と入力すると、結果の中に `cpe:2.3:a:libpng:libpng:...` のような文字列が表示されます。`cpe:2.3:a:` の次がベンダー、その次が製品名です。部品名で何も出ない場合は、別の書き方（例：`libxml` と `libxml2`、`expat` と `libexpat`）でも試します。

よく使われる部品の例を示します。必ず上のコマンドで最新の表記を確認してください。

部品	CPEベンダー	CPE製品名
zlib	zlib	zlib
OpenSSL	openssl	openssl
libpng	libpng	libpng
libxml2	xmlsoft	libxml2
Expat	libexpat_project	libexpat
curl / libcurl	haxx	curl（libcurl の場合は libcurl も確認）
SQLite	sqlite	sqlite
FreeType	freetype	freetype

どうしても見つからない部品は、データベースにまだ脆弱性が登録されていない可能性があります。その場合はCPEベンダー欄に「なし」と書き、備考欄に「CPE該当なし（確認日）」と書いておきます。

記号を含むCPE。 CPEには `paho_mqtt_c\/c\+\+_client`（Eclipse Paho のC/C++クライアント）のように \ 付きの記号を含むものがあります。台帳には \ を取り除いた形（`paho_mqtt_c/c++_client`）で書いてください。`cpp_sbom convert` が自動で \ を付け直します。

データベース以外での調べ方。 ネット接続PCで、NVDの CPE検索ページ に部品名を入力して調べることもできます。入力するのは公開されているOSSの名前だけなので、社内の情報は外に出ません。

付録C 配布する実行ファイル

次の2つは、本書のために作成し、サンプル（paho.mqtt と nlohmann-json を使うMQTTクライアント）で動作を確認した実行ファイルです。本書の管理者から受け取ってください。どちらも単体で動き、外部との通信は一切行いません。

ツール	使う場所と作業	手順で使うファイル名
cpp_deps	ビルドPC：作業1	`cpp_deps-linux-amd64`（Linux）、`cpp_deps-windows-amd64.exe`（Windows）
cpp_sbom	作業PC：第5章または第6章（`ledger` 台帳を作る、`check` 完成を確かめる、`convert` SBOMに変換する）	`cpp_sbom-windows-amd64.exe`

本書の手順では作業PCを Windows としていますが、配布物には Linux 版の `cpp_sbom-linux-amd64` も含まれています。どちらのツールにも、aarch64 の Linux 向け（`-linux-arm64`）があります。

システム製品がWindowsの場合は、配布物の `windows-system-inventory.ps1` を完成した納入構成で実行します。PowerShell標準機能だけを使い、インストールや外部通信はしません。

cpp_deps がしていること

cpp_deps はビルドPCで動かすツールなので、何をしているかを示します。**ビルドで使った部品を調べて、結果を `deps.json` に書き出すだけ**のツールです。

	Linux	Windows
読むもの	`compile_commands.json`、ビルドログ、成果物の実行ファイルとライブラリ、`/etc/os-release`	ビルドログ、成果物のEXE/DLL
実行するコマンド	コンパイラ（`-M` を付けて、読み込むヘッダの一覧を出すだけ）、`ldd`（使うライブラリの一覧）、`dpkg` と `dpkg-query` または `rpm`（持ち主のパッケージの確認）	なし
書き出すもの	`--out` に指定したファイル（`deps.json`）だけ	同じ

次のことはしません。

- **外部との通信。** 通信の機能を組み込んでいません。
- **ソースや成果物の変更、ビルド環境の変更。** インストールも不要で、設定も残しません。

`deps.json` に入るのは、ファイルのパス、パッケージの名前と版、EXE/DLLのバージョン情報（製品名、会社名、著作権表示など）です。**ソースコードの本文は入りません。** テキストファイルなので、メモ帳などで中身を確認できます。

付録D 困ったとき

症状	原因と対処
grype が「database … could not be loaded」等のエラーで止まる	データベースが取り込まれていません。3.3 の「脆弱性データベースを取り込む」を実行してください
grype db import が「failed to move database directory」「being used by another process」と表示して失敗する	Grype の版が古い可能性があります。0.121.2 以上に入れ替えてから取り込み直してください（検証では 0.119.0 で多発し、0.121.2 では発生しませんでした）
grype の起動に時間がかかり、通信エラーが出る	「ネットにつながない設定」のファイルが作られていないか、名前が違います。%USERPROFILE%\.grype.yaml を確認してください
scancode --help で「'py' is not recognized」と出る	Python のランチャー（py）が入っていません。$env:PYTHON_EXECUTABLE = "python" を実行してから、C:\sbom-tools\scancode-toolkit-<版>\configure.bat を実行してください
ScanCode の初回設定で「ファイル名または拡張子が長すぎます」と出る	ScanCode を長いパスの中に置いています。C:\sbom-tools の直下に置き直し、できた venv フォルダを削除してやり直してください
cpp_deps で「コンパイル命令は再実行できませんでした」と出る	ビルドしたPCと別のPCで実行しているか、ビルド後にフォルダを移動しています。ビルドPCの、同じフォルダで実行してください
cpp_deps-linux-amd64 などで「許可がありません」「Permission denied」と出る	実行の許可が付いていません。1.3 の chmod +x の行を実行してください
Windows の build.log にインクルードやリンクの記録が出ない（ツールが「注意:」を表示する）	環境変数の設定が効いていません。2.2 (2) の設定を、msbuild と同じ画面で実行したか確認してください。それでも出ない場合は、プロジェクトのプロパティで [C/C++] → [詳細設定] → [インクルードの表示] を [はい]、[リンカー] → [全般] → [進行状況の表示] を [すべての進行状況メッセージを表示する] にしてビルドし直してください。CMake の場合は、2.2 (2) の .sln か .slnx についての説明を確認してください
cpp_deps で「MSVC 以外のコンパイラ（MinGW / clang など）でビルドされているようです」と出る	この手順の対象外です。ヘッダと静的ライブラリを調べられないので、本書の管理者に相談してください
tar で展開するときに、ファイルを作れないというエラーが出る	Linux のファイル名に Windows で使えない文字（: * ? など）が含まれています。そのファイルは ScanCode で調べられないので、開発担当者に内容を確認してもらってください
cpp_sbom convert で「CPEもPURLもありません」と出る	CPEベンダーに「なし」と書いた部品で出ます。この部品は脆弱性の照合で見つからない可能性があります。付録Bで調べ直してもCPEがなければ、そのままで構いません
grype の結果で、名前の似た別の製品の脆弱性が出る	CPEによる照合で起きる誤検知です。8.3 の手順で確認し、8.4 の方法で判断を記録してください
cve-bin-tool の取得で gsutil に関するエラー（FileNotFoundError）が出る	-d "OSV,GAD" が付いていません。3.2 (5) のコマンドのとおりに実行してください
cve-bin-tool の実行後に結果ファイルがない	データが取り込まれていません（画面に offline operation の案内が出ます）。3.3 の「脆弱性データベースを取り込む」を実行してください
cve-bin-tool の --import で FileNotFoundError と出る	取り込み先のフォルダがありません。3.3 の New-Item の行を先に実行してください

症状	原因と対処
cpp_sbom ledger が「上書きしません」と表示して止まる	同じ名前の台帳がすでにあります。記入した内容を守るための動作です。作り直すときは、別の名前を --out に指定するか、古いファイルを移動してください
cpp_sbom check が「作成条件」シートがないと表示する	旧方式の台帳です。納入形態が記録されていないため、第5章または第6章で製品種別を指定して台帳を作り直してください
cpp_sbom ledger が「ScanCode で調べたフォルダと –src が違います」と表示して止まる	作業3で ScanCode に指定したフォルダと同じものを --src に指定してください
cpp_sbom ledger が「Linux の結果には –osenv … を指定してください」と表示して止まる	Linux 向けの製品です。ビルドPCから運んだ build-env.cdx.json を --osenv に指定してください
cpp_sbom check が「要確認: …: 台帳に見当たりません」「バージョンが台帳と違います」と表示して止まる	作業4で見つかった部品が台帳の行と結び付きません。第5章または第6章の「要確認」の説明のとおり、行を追加するか、バージョンを直すか、「対象外」の行を追加してください
cpp_sbom check が「cve-bin-tool が作った結果ではありません」と表示して止まる	--cvebin に別のファイルを指定しています。作業4で作った cvebin-parts.json を指定してください
cpp_sbom check や cpp_sbom convert が「台帳が Excel で開かれているようです」と表示する	保存していない変更は読み取られません。台帳を保存して Excel を閉じてから、再実行してください
cpp_sbom check や cpp_sbom convert が「12行目（同梱）: ライセンスが未記入です」のように表示して止まる	台帳に未記入の項目が残っています。表示された行の色の付いたセルを記入し、保存してから再実行してください